Super Mart Grocery Sales - Retail Analytics Dataset

Objective:

This project focuses on using a dataset containing information about grocery sales at a supermarket. The dataset includes columns such as Order ID, Customer Name, Category, Sub Category, City, Order Date, Region, Sales, Discount, Profit, State, month_no, Month, and year. We'll explore this data, perform feature engineering, and build a machine learning model to predict sales or profit.

Step 1: Import Required Libraries

First, let's import the necessary libraries for data manipulation, visualization, and machine learning.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

Step 2: Load the Dataset

Assume we have a dataset named supermart_grocery_sales.csv. Let's load the data into a pandas DataFrame.

```
# Load the dataset
data = pd.read_csv('supermart_grocery_sales.csv')
```

Super Mart Grocery Sales - Retail Analytics Dataset

```
# Display the first few rows of the dataset
print(data.head())
```

Step 3: Data Preprocessing

1. Check for Missing Values and Handle Them

```
# Check for missing values
print(data.isnull().sum())

# Drop any rows with missing values
data.dropna(inplace=True)

# Check for duplicates
data.drop_duplicates(inplace=True)
```

2. Convert Date Columns to DateTime Format

```
# Convert 'Order Date' to datetime format
data['Order Date'] = pd.to_datetime(data['Order Date'])

# Extract day, month, and year from 'Order Date'
data['Order Day'] = data['Order Date'].dt.day
data['Order Month'] = data['Order Date'].dt.month
data['Order Year'] = data['Order Date'].dt.year
```

Super Mart Grocery Sales - Retail Analytics Dataset

3. Label Encoding for Categorical Variables

Convert categorical variables such as Category, Sub Category, City, Region,

State, and Month into numerical values.

```
# Initialize the label encoder
le = LabelEncoder()

# Encode categorical variables
data['Category'] = le.fit_transform(data['Category'])
data['Sub Category'] = le.fit_transform(data['Sub Category'])
data['City'] = le.fit_transform(data['City'])
data['Region'] = le.fit_transform(data['Region'])
data['State'] = le.fit_transform(data['State'])
data['Month'] = le.fit_transform(data['Month'])

# Display the first few rows after encoding
print(data.head())
```

Step 4: Exploratory Data Analysis (EDA)

1. Distribution of Sales by Category

```
plt.figure(figsize=(10, 6))
sns.boxplot(x='Category', y='Sales', data=data, palette='Set2')
plt.title('Sales Distribution by Category')
plt.xlabel('Category')
plt.ylabel('Sales')
plt.show()
```

Super Mart Grocery Sales - Retail Analytics Dataset

2. Sales Trends Over Time

```
plt.figure(figsize=(12, 6))

data.groupby('Order Date')['Sales'].sum().plot()

plt.title('Total Sales Over Time')

plt.xlabel('Date')

plt.ylabel('Total Sales')

plt.show()
```

3. Correlation Heatmap

```
plt.figure(figsize=(12, 6))

corr_matrix = data.corr()

sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')

plt.title('Correlation Heatmap')

plt.show()
```

Step 5: Feature Selection and Model Building

We'll use features like Category, Sub Category, City, Region, State,

month_no, Discount, and Profit to predict Sales.

```
# Select features and target variable

features = data.drop(columns=['Order ID', 'Customer Name',

'Order Date', 'Sales', 'Month'])

target = data['Sales']


# Split the data into training and testing sets
```

Super Mart Grocery Sales - Retail Analytics Dataset

```
X_train, X_test, y_train, y_test = train_test_split(features,

target, test_size=0.2, random_state=42)


# Feature scaling

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)
```

Step 6: Train a Linear Regression Model

```
# Initialize the model

model = LinearRegression()


# Train the model


model.fit(X_train, y_train)


# Make predictions

y_pred = model.predict(X_test)
```

Step 7: Evaluate the Model

Evaluate the model performance using Mean Squared Error (MSE) and R-squared.

```
# Calculate MSE and R-squared

mse = mean_squared_error(y_test, y_pred)

r2 = r2_score(y_test, y_pred)
```

```
print(f'Mean Squared Error: {mse}')
print(f'R-squared: {r2}')
```

Sample Output:

Mean Squared Error: 1758.26

R-squared: 0.82

Step 8: Visualize the Results

1. Actual vs Predicted Sales

```
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred)
plt.plot([min(y_test), max(y_test)], [min(y_test),

max(y_test)], color='red')
plt.title('Actual vs Predicted Sales')
plt.xlabel('Actual Sales')
plt.ylabel('Predicted Sales')
plt.show()
```

Step 9: Conclusion

- The linear regression model provided a reasonable prediction for sales based on the features selected.
- The model’s R-squared value indicates a good fit, explaining a significant portion of the variance in sales.

Super Mart Grocery Sales - Retail Analytics Dataset

● Further refinement of the model could involve trying different machine learning algorithms, such as decision trees or ensemble methods.

Next Steps:

1. Advanced Modeling: Experiment with more complex models like Random Forest or XGBoost to improve predictions.
2. Feature Engineering: Explore additional features or interactions between features to enhance model performance.
3. Model Deployment: Integrate the model into a dashboard for real-time sales prediction and business analytics.

This project provides a hands-on introduction to data analysis and machine learning for beginners, with a focus on retail sales data.

About Dataset

This is a fictional dataset created for helping the data analysts to practice exploratory data analysis and data visualization. The dataset has data on orders placed by customers on a grocery delivery application.

The dataset is designed with an assumption that the orders are placed by customers living in the state of Tamil Nadu, India.